BDD SQL — Examen blanc : corrigé

BUT3 GEA — IUT de Saint-Denis

Rappel des données :

services

id	nom
1	Audit
2	Conseil
3	Formation

consultants

id	nom	ville	service_id	tarif_jour
1	Martin	Paris	1	650
2	Diallo	Lyon	2	800
3	Nguyen	Paris	3	450
4	Bernard	Lille	1	700
5	Haddad	Paris	2	900
6	Petit	Lyon	3	500
7	Moreau	Marseille	2	750
8	Martin	Lille	3	480

clients

id	nom	secteur	ville
1	Boulangeries Durand	Commerce	Paris
2	Transports Leroy	Transport	Lyon
3	Mairie de Bobigny	Public	Bobigny
4	Clinique Saint-Jean	Santé	Lille
5	Textiles Moulin	Industrie	Roubaix
6	Banque Horizon	Finance	Paris

missions

id	consultant_id	client_id	annee	jours
1	1	1	2024	5
2	2	2	2024	10
3	3	3	2025	3
4	1	4	2025	8
5	5	6	2025	12
6	4	4	2024	6
7	2	6	2025	4
8	6	3	2025	2
9	7	2	2025	7
10	5	1	2024	3
11	1	6	2025	9
12	8	5	2025	4
13	3	5	2024	6
14	2	1	2025	5
15	4	6	2025	10

Exercice 1

1. `SELECT * FROM consultants WHERE ville = "paris";` → **aucune** — `=` tient compte des majuscules : “paris” ≠ “Paris”.
2. `SELECT * FROM consultants WHERE nom LIKE "%a%" AND tarif_jour > 600;` → **1, 2, 4, 5, 7**
3. `SELECT * FROM consultants WHERE ville LIKE "L%" OR tarif_jour >= 800;` → **2, 4, 5, 6, 8**
4. `SELECT * FROM consultants WHERE (ville = "Lyon" OR ville = "Lille") AND service_id = 3;` → **6, 8** — les parenthèses s’appliquent d’abord.
5. `SELECT * FROM consultants WHERE ville = "Lyon" OR ville = "Lille" AND service_id = 3;` → **2, 6, 8** — sans parenthèses, `AND` passe avant `OR` : on obtient « Lyon » OU « (Lille ET Formation) ». Comparez avec la requête 4.
6. `SELECT * FROM consultants WHERE nom LIKE "_a%" AND ville NOT IN ("Paris", "Lyon");` → **8** — `_a%` : « a » en 2e lettre (Martin, Haddad) ; puis on exclut Paris et Lyon.

Exercice 2

1. `SELECT ville, COUNT(*) AS nb FROM consultants GROUP BY ville ORDER BY nb DESC, ville;`

ville	nb
Paris	3
Lille	2
Lyon	2
Marseille	1

2. `SELECT service_id, MIN(tarif_jour) AS mini, MAX(tarif_jour) AS maxi FROM consultants GROUP BY service_id;`

service_id	mini	maxi
1	650	700
2	750	900
3	450	500

3. `SELECT annee, SUM(jours) AS total_jours FROM missions WHERE jours >= 5 GROUP BY annee;`

annee	total_jours
2024	27
2025	51

4. `SELECT consultant_id, COUNT(*) AS nb FROM missions GROUP BY consultant_id HAVING COUNT(*) >= 3;`

consultant_id	nb
1	3
2	3

5. `SELECT consultants.nom, SUM(missions.jours) AS jours FROM missions JOIN consultants ON missions.consultant_id = consultants.id GROUP BY consultants.nom HAVING SUM(missions.jours) > 12;`

nom	jours
Bernard	16
Diallo	19
Haddad	15
Martin	26

Pourquoi la requête 5 est trompeuse. Elle regroupe par **nom**, or deux consultants différents s'appellent Martin (id 1 à Paris, 22 jours ; id 8 à Lille, 4 jours). Leurs jours sont additionnés (26) comme s'il s'agissait d'une seule personne. Il faut regrouper par la clé primaire : `GROUP BY consultants.id, consultants.nom`. On obtient alors Martin (id 1) avec 22 jours, et Martin (id 8) disparaît (4 jours, pas plus de 12).

```sql
SELECT consultants.id, consultants.nom, SUM(missions.jours) AS jours FROM missions JOIN consultants ON missions.consultant_id = consultants.id GROUP BY consultants.id, consultants.nom HAVING SUM(missions.jours) > 12;
```

id	nom	jours
1	Martin	22
2	Diallo	19
4	Bernard	16
5	Haddad	15

Exercice 3

1. `SELECT nom ville FROM consultants WHERE tarif_jour > 600;`

S'exécute, mais **ne fait pas ce qu'on veut** : il manque une virgule. `nom ville` signifie « la colonne nom, renommée ville » : on obtient une seule colonne, intitulée ville, qui contient les noms.

```sql
SELECT nom, ville FROM consultants WHERE tarif_jour > 600;
```

2. `SELECT ville, COUNT(*) FROM consultants WHERE COUNT(*) > 1 GROUP BY ville;`

   Erreur (*misuse of aggregate*) : on ne peut pas utiliser une fonction d'agrégation dans `WHERE`. Pour filtrer des groupes, c'est `HAVING`, après `GROUP BY`.

   ```sql
   SELECT ville, COUNT(*) FROM consultants GROUP BY ville HAVING COUNT(*) > 1;
   ```

3. `SELECT nom, tarif_jour * 1,1 AS nouveau_tarif FROM consultants;`

   S'exécute, mais **ne fait pas ce qu'on veut** : la virgule sépare deux colonnes. On obtient `tarif_jour * 1` et une colonne `nouveau_tarif` qui vaut 1 partout. Les décimaux s'écrivent avec un point.

   ```sql
   SELECT nom, tarif_jour * 1.1 AS nouveau_tarif FROM consultants;
   ```

4. `SELECT clients.nom, missions.jours FROM missions JOIN clients;`

   S'exécute, mais **ne fait pas ce qu'on veut** : il manque la condition `ON`. Chaque mission est associée à **chaque** client : 15 × 6 = 90 lignes sans signification.

   ```sql
   SELECT clients.nom, missions.jours FROM missions JOIN clients ON missions.client_id = clients.id;
   ```

5. `SELECT service_id, COUNT(*) FROM consultants HAVING COUNT(*) > 2 GROUP BY service_id;`

   Erreur de syntaxe : l'ordre des clauses est fixe, `GROUP BY` vient avant `HAVING`.

   ```sql
   SELECT service_id, COUNT(*) FROM consultants GROUP BY service_id HAVING COUNT(*) > 2;
   ```

6. `SELECT ville, nom, AVG(tarif_jour) FROM consultants GROUP BY ville;`

   S'exécute (en SQLite), mais **le résultat n'a pas de sens** : `nom` n'est ni dans le `GROUP BY` ni dans une fonction d'agrégation. Il y a plusieurs consultants par ville, SQLite en affiche un au hasard. On retire `nom` (ou on regroupe aussi par nom, mais alors on n'a plus une moyenne par ville).

   ```sql
   SELECT ville, AVG(tarif_jour) FROM consultants GROUP BY ville;
   ```

7. `SELECT * FROM missions WHERE annee = 2025 ORDER BY jours DESC;`

   Correcte : les missions de 2025, de la plus longue à la plus courte.

Exercice 4

D'autres écritures sont possibles (alias, ordre des colonnes…) : ce qui compte, c'est que la requête donne le bon résultat. Les résultats sont indiqués pour vous permettre de vérifier.

1. Le nom et le tarif des consultants dont le tarif journalier est compris entre 500 et 750 (inclus), du plus cher au moins cher.

```sql
SELECT nom, tarif_jour FROM consultants WHERE tarif_jour >= 500 AND tarif_jour <= 750 ORDER BY tarif_jour DESC;
```

nom	tarif_jour
Moreau	750
Bernard	700

Martin	650
Petit	500

2. Le nombre de missions par année.

```
SELECT annee, COUNT(*) AS nb_missions FROM missions GROUP BY
annee;
```

annee	nb_missions
2024	5
2025	10

3. Pour chaque mission : le nom du consultant, le nom du client et le nombre de jours.

```
SELECT consultants.nom, clients.nom, missions.jours FROM missions
JOIN consultants ON missions.consultant_id = consultants.id JOIN
clients ON missions.client_id = clients.id;
```

nom	nom	jours
Martin	Boulangeries Durand	5
Diallo	Transports Leroy	10
Nguyen	Mairie de Bobigny	3
Martin	Clinique Saint-Jean	8
Haddad	Banque Horizon	12
Bernard	Clinique Saint-Jean	6
Diallo	Banque Horizon	4
Petit	Mairie de Bobigny	2
Moreau	Transports Leroy	7
Haddad	Boulangeries Durand	3
Martin	Banque Horizon	9
Martin	Textiles Moulin	4
Nguyen	Textiles Moulin	6
Diallo	Boulangeries Durand	5
Bernard	Banque Horizon	10

4. Pour chaque mission : son id, le nom du consultant et le montant facturé (nombre de jours × tarif journalier du consultant), dans une colonne nommée montant, du plus grand au plus petit montant.

```
SELECT missions.id, consultants.nom, missions.jours *
consultants.tarif_jour AS montant FROM missions JOIN consultants
ON missions.consultant_id = consultants.id ORDER BY montant DESC;
```

id	nom	montant
5	Haddad	10800
2	Diallo	8000
15	Bernard	7000
11	Martin	5850
9	Moreau	5250
4	Martin	5200
6	Bernard	4200
14	Diallo	4000
1	Martin	3250
7	Diallo	3200
10	Haddad	2700

13	Nguyen	2700
12	Martin	1920
3	Nguyen	1350
8	Petit	1000

5. Le chiffre d'affaires total de chaque consultant (id, nom, total des montants facturés), du plus grand au plus petit. Attention : deux consultants portent le même nom.

```
SELECT consultants.id, consultants.nom, SUM(missions.jours *
consultants.tarif_jour) AS ca FROM missions JOIN consultants ON
missions.consultant_id = consultants.id GROUP BY consultants.id,
consultants.nom ORDER BY ca DESC;
```

id	nom	ca
2	Diallo	15200
1	Martin	14300
5	Haddad	13500
4	Bernard	11200
7	Moreau	5250
3	Nguyen	4050
8	Martin	1920
6	Petit	1000

Regrouper par `consultants.id` : sinon les deux Martin sont additionnés.

6. Le nom des clients et leur nombre total de jours de mission, en ne gardant que les clients qui totalisent au moins 10 jours.

```
SELECT clients.nom, SUM(missions.jours) AS total_jours FROM
missions JOIN clients ON missions.client_id = clients.id GROUP BY
clients.id, clients.nom HAVING SUM(missions.jours) >= 10;
```

nom	total_jours
Boulangeries Durand	13
Transports Leroy	17
Clinique Saint-Jean	14
Textiles Moulin	10
Banque Horizon	35

Regrouper par `clients.id` (deux clients pourraient avoir le même nom) ; `HAVING` car la condition porte sur une somme.

7. Pour chaque service (par son nom) : le nombre de missions réalisées par ses consultants et le chiffre d'affaires correspondant.

```
SELECT services.nom, COUNT(*) AS nb_missions, SUM(missions.jours
* consultants.tarif_jour) AS ca FROM missions JOIN consultants ON
missions.consultant_id = consultants.id JOIN services ON
consultants.service_id = services.id GROUP BY services.id,
services.nom;
```

nom	nb_missions	ca
Audit	5	25500
Conseil	6	33950
Formation	4	6970

Trois tables : missions → consultants → services.

8. Le chiffre d'affaires réalisé en 2025 par ville des consultants, en ne gardant que les villes où il dépasse 10 000.

```sql
SELECT consultants.ville, SUM(missions.jours *
consultants.tarif_jour) AS ca FROM missions JOIN consultants ON
missions.consultant_id = consultants.id WHERE missions.annee =
2025 GROUP BY consultants.ville HAVING SUM(missions.jours *
consultants.tarif_jour) > 10000;
```

ville	ca
Paris	23200

`WHERE` pour l'année (sur les lignes, avant les groupes), `HAVING` pour le seuil (sur les groupes). Lille fait 8 920 et Lyon 8 200 : seule Paris dépasse 10 000.

9. Pour chaque secteur d'activité des clients : le nombre moyen de jours par mission, arrondi à 1 chiffre après la virgule.

```sql
SELECT clients.secteur, ROUND(AVG(missions.jours), 1) AS
jours_moyens FROM missions JOIN clients ON missions.client_id =
clients.id GROUP BY clients.secteur;
```

secteur	jours_moyens
Commerce	4.3
Finance	8.8
Industrie	5
Public	2.5
Santé	7
Transport	8.5

10. Le nom des clients (sans doublon, par ordre alphabétique) qui ont fait appel à au moins un consultant du service « Formation ». (Quatre tables.)

```sql
SELECT DISTINCT clients.nom FROM missions JOIN clients ON
missions.client_id = clients.id JOIN consultants ON
missions.consultant_id = consultants.id JOIN services ON
consultants.service_id = services.id WHERE services.nom =
'Formation' ORDER BY clients.nom;
```

nom
Mairie de Bobigny
Textiles Moulin

Quatre tables. `DISTINCT` car un même client peut avoir eu plusieurs missions de ce service. Alternative acceptée : `WHERE consultants.service_id = 3` (trois tables seulement).

Exercice 5

1.

```sql
CREATE TABLE evaluations (
    id          INTEGER PRIMARY KEY,
    mission_id  INTEGER REFERENCES missions(id),
    note        INTEGER NOT NULL,
    commentaire TEXT
);
```

2. Pour la deuxième, on peut aussi ne pas mentionner `commentaire` du tout.

```sql
INSERT INTO evaluations (id, mission_id, note, commentaire)
VALUES
    (1, 5, 4, 'Très bien'),
    (2, 8, 2, NULL);
```

3.

```sql
UPDATE consultants SET tarif_jour = ROUND(tarif_jour * 1.05, 2)
WHERE service_id = 2;
```

Nouveaux tarifs : Diallo 840, Haddad 945, Moreau 787.5.

4.

```sql
DELETE FROM missions WHERE annee = 2024;
```

Il reste **10** missions (15 − 5 missions de 2024).

5. Sans `WHERE`, elle s'applique à **toutes** les lignes : tous les consultants ont désormais Paris comme ville.

Exercice 6

`WHERE` filtre les **lignes**, avant que les groupes soient faits ; `HAVING` filtre les **groupes**, après le `GROUP BY`, et peut donc utiliser une fonction d'agrégation (`HAVING COUNT(*) > 2`), ce que `WHERE` ne peut pas.